

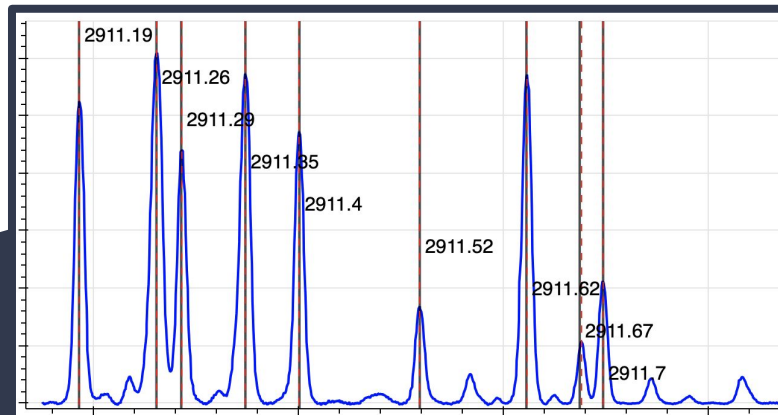
Xpectra - A Python Spectral Analysis Package for Modeling Laboratory Spectra [beta version]

Emily Mader (STT)

Team: Ehsan (Sam) Gharib-Nezhad (STT), Salma Bejaoui (STA), Natasha Batalha (STA), Richard Freedman (STT), Farid Salama (STA), Iouli Gordon (Harvard)

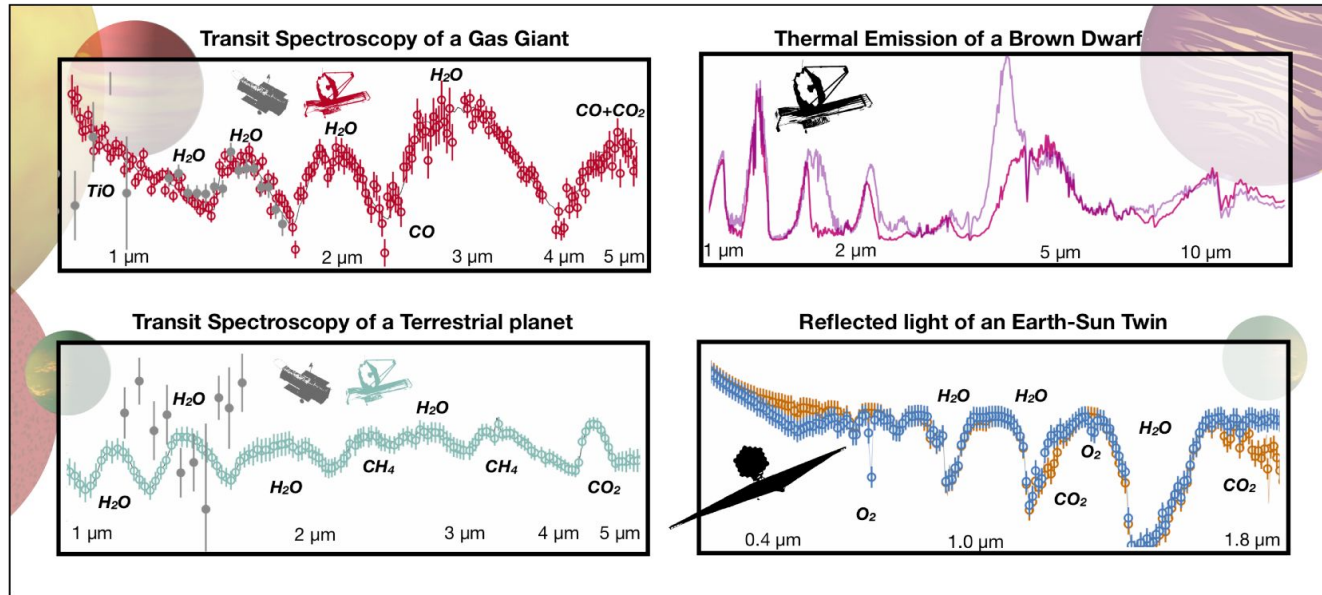
Final Internship Deliverable

Dec 12, 2024



We care about Opacity data!!!

- Opacities determine how molecules/atoms absorb light under different pressure & temperature conditions

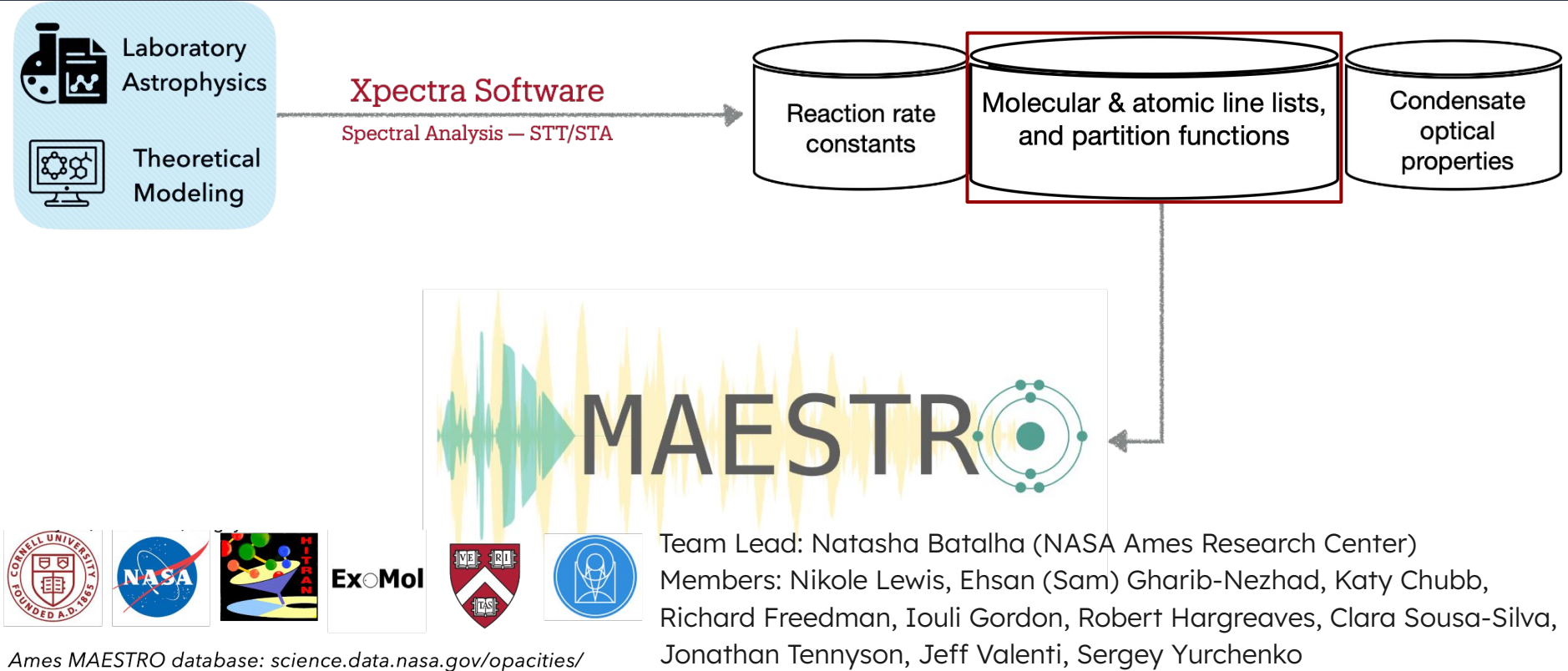


- Opacities are used as input to all atmosphere models

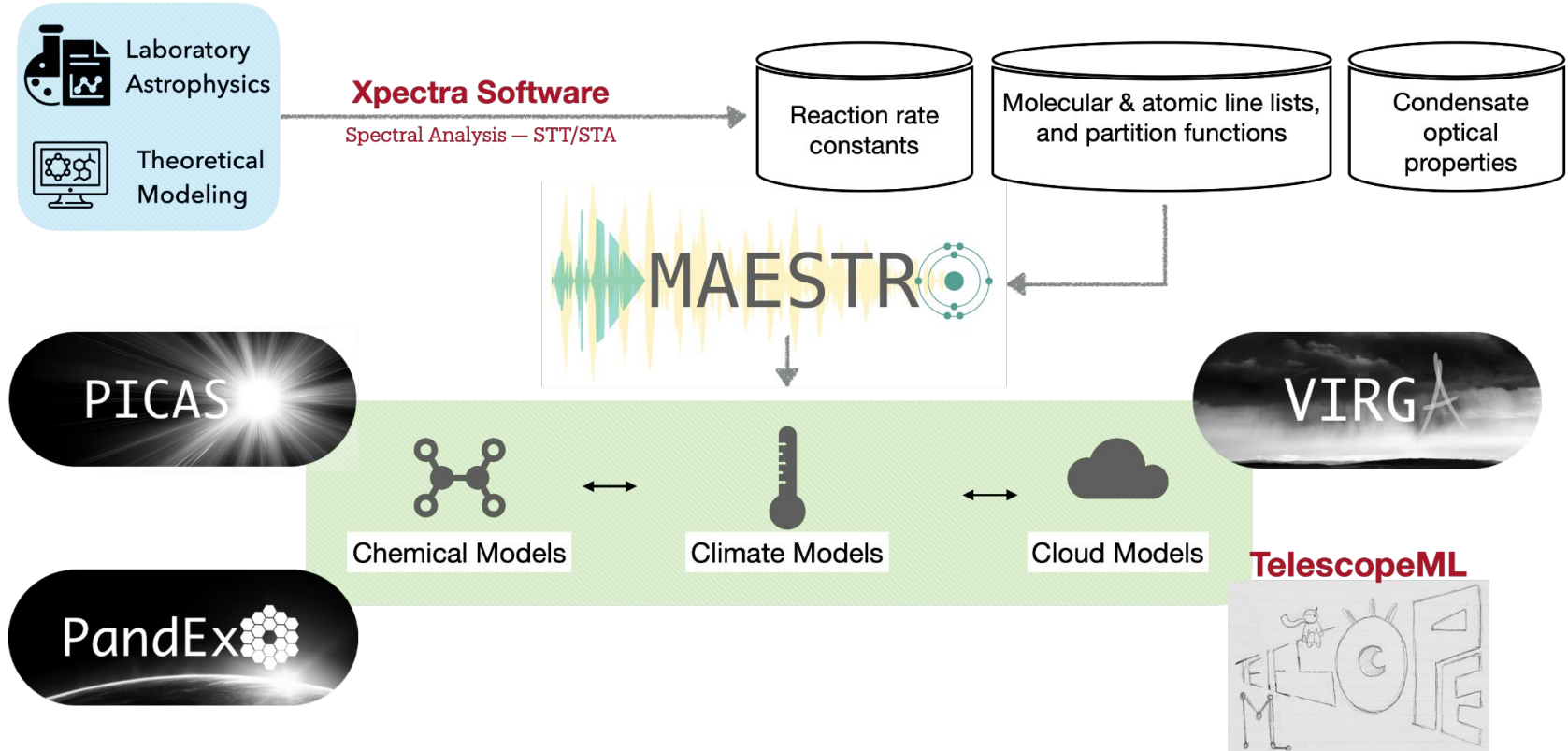
powered by

PICAS

Xpectra initiative in Ames Exoplanet Ecosystem

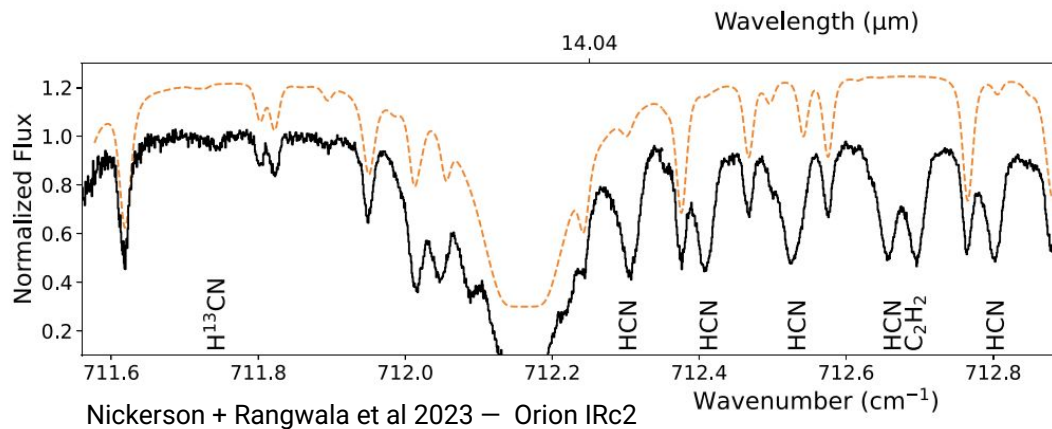


Xpectra initiative in Ames Exoplanet Ecosystem



Potential Impact of Xpectra on code ST Research:

- **Salma Bejaoui / Farid Salam:** Fitting Laser spectra
- **Naseem Rangwala / Sarah Nickerson:** Assign lines from Galaxy data
- **Ella Sciamma-O'Brien / Joe Roser:** Optical constants
- **Michel Nuevo:** UV-Vis
- **Nikki Parenteau:** Line list for HWO
- **Exo folks:** Opacity and beyond

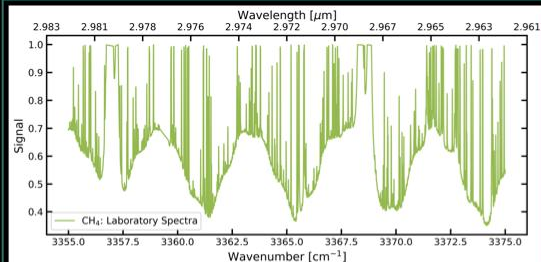


Xpectra: Spectral Analysis

(1) Lab Spectrum

→ Prepare spectrum

→ Define wavenumber range

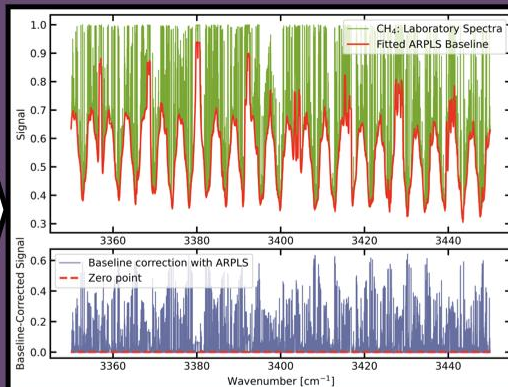


→ Interactive plotting

→ Explore spectral features

(2) Baseline Correction

→ Model baseline with function



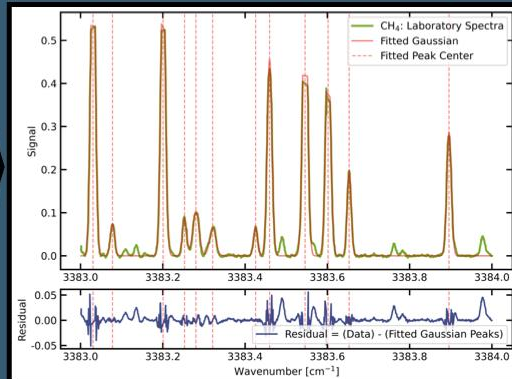
→ Fit baseline, apply correction

→ Compare correction methods

(3) Fit Spectral Peaks

→ Find peaks automatically/manually

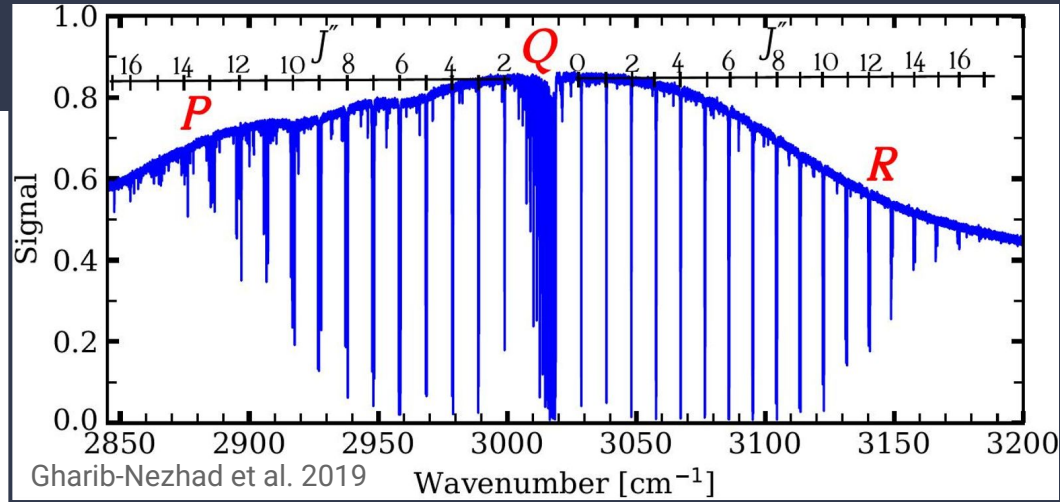
→ Assign lines using HITRAN



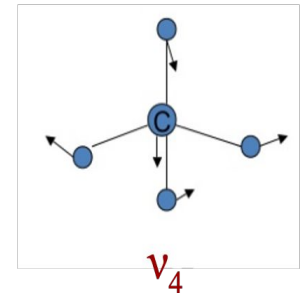
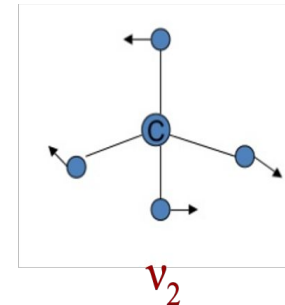
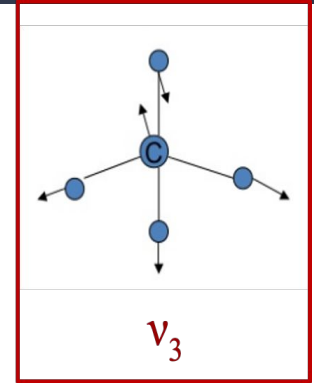
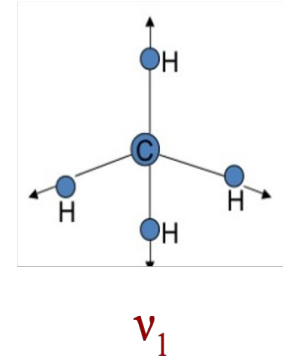
→ Model peaks with line profile

→ Extract pressure broadening coeff.

Case study: CH₄



- Important absorber in exoplanetary and brown dwarf atmospheres
- Vibrational modes
 - ◆ Stretching: ν_1 (symmetric), ν_3 (asymmetric)
 - ◆ Bending: ν_2 , ν_4

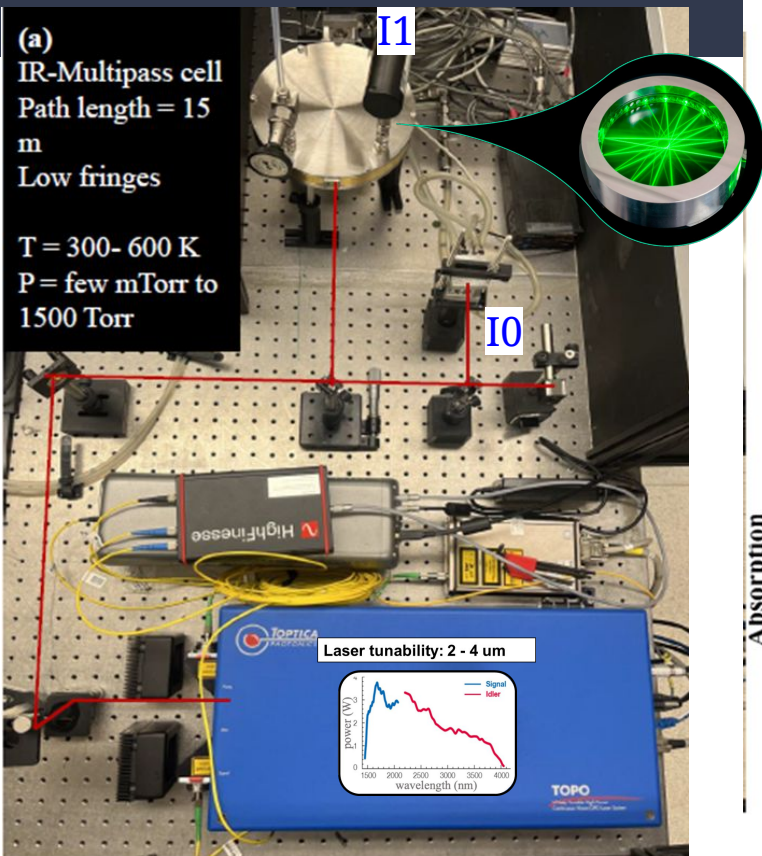


Pressure Broadening Lab Measurements: Overview [1/2]

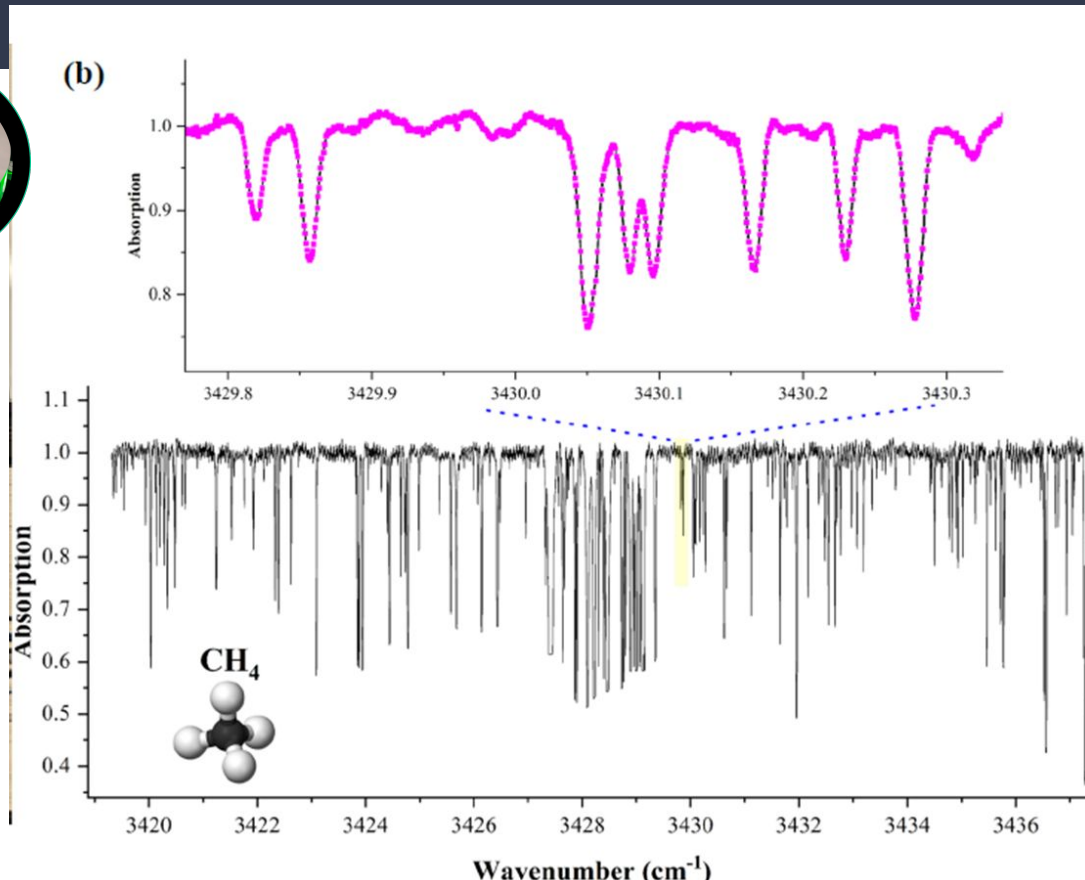
Multipass gas cell (300-600 K)

(a)
IR-Multipass cell
Path length = 15 m
Low fringes

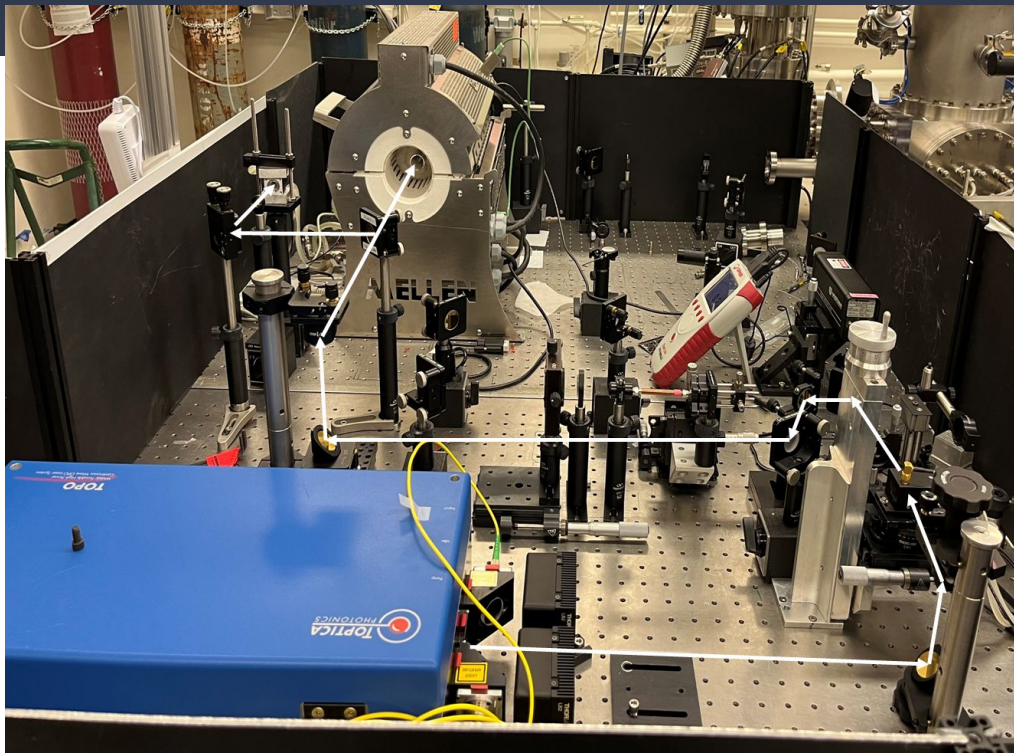
$T = 300 - 600 \text{ K}$
 $P = \text{few mTorr to } 1500 \text{ Torr}$



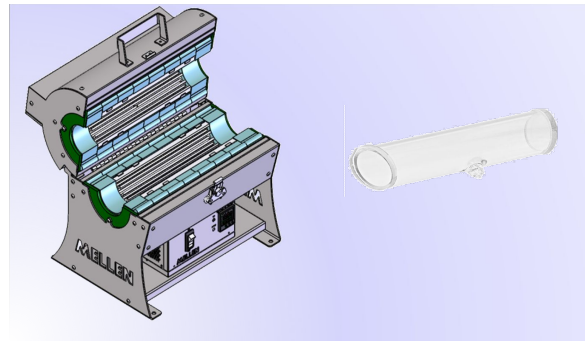
Lab Task Lead: Salma Bejaoui



Pressure Broadening Lab Measurements: Overview [2/2]

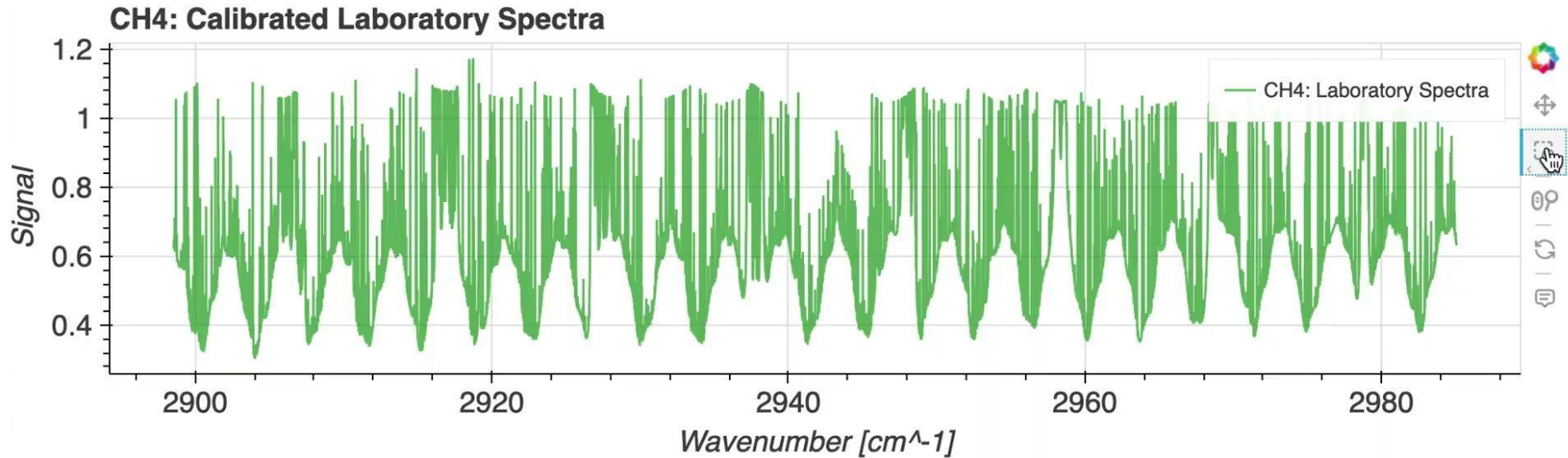


Temperature range: 300-1250 K



				<i>Broadeners</i>		
		λ (<i>um</i>)	Intensity	H ₂	He	CO ₂
<i>Absorbers</i>	CO ₂	2.68	4 10 ⁻²⁰	✓	✓	✓
		2.77	6 10 ⁻²⁰	✓	✓	✓
	CO	2.35	4 10 ⁻²¹	✓	✓	✓
	CH ₄	3.3	2 10 ⁻¹⁹		✓	✓
	HCN	3.03	3 10 ⁻¹⁹	✓	✓	

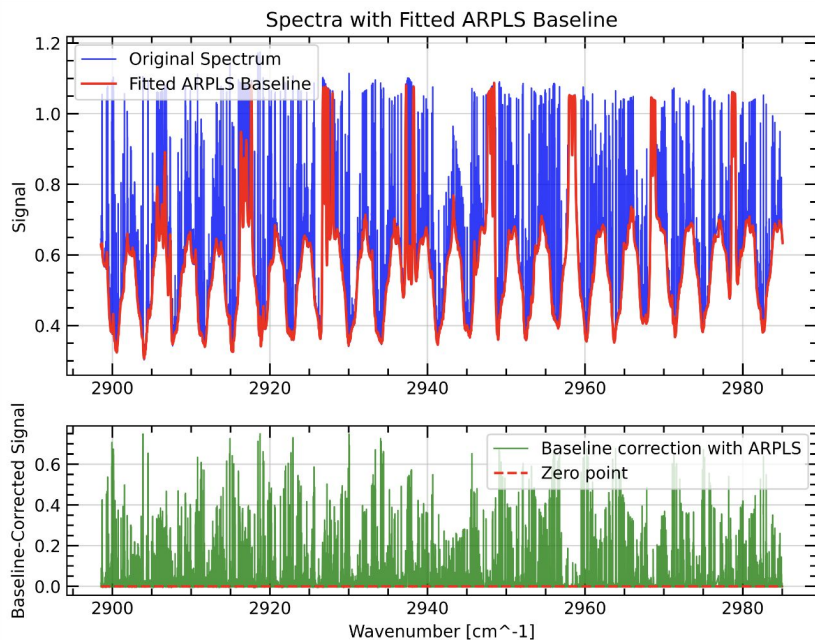
Example of Laboratory Spectra: CH₄ 100 Torr



Baseline Correction

→ Run ARPLS baseline correction, visualize the plot, and save the plot as a pdf

```
[8]: # Fit baseline using ARPLS algorithm
specfit.arpls(__plot__ = True,
              __save_plots__ = True,
              __print__ = True)
```



```
[9]: specfit.baseline_type
```

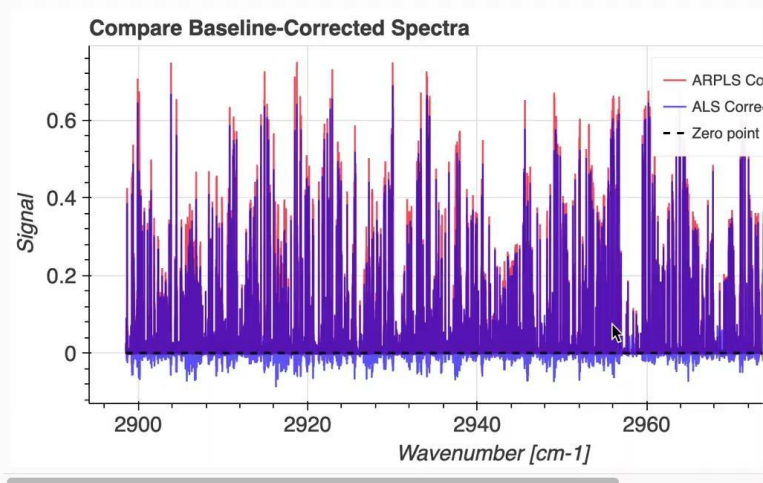
```
[9]: 'arpls'
```

→ Let's perform qualitative analysis on the baseline-correction methods by overplotting the residuals from ARPLS and ALS methods.

```
[12]: plot_compare_baselines(wavenumber_values = x,
                             corrected_signal_1 = specfit.y_baseline_corrected_ARPLS,
                             baseline_type_1 = 'ARPLS',
                             corrected_signal_2 = specfit.y_baseline_corrected_ALS,
                             baseline_type_2 = 'ALS'
                             )
```



BokehJS 3.7.0-dev.1 successfully loaded.



Algorithm Reference: Baek, et al. Analyst, 2015, 140, 250 (2015)

Parsing HITRAN Linelist

HITRAN Database Format 2004-present

```
[5]: # Parse file to DataFrame
assign.parse_file_to_dataframe()
```

```
[5]:
```

	molec_id	local_iso_id	nu	sw	a	gamma_air	gamma_self	elower	n_air
0	6	2	2900.000621	1.825000e-25	0.023890	0.0490	0.067	814.6845	0.63
1	6	2	2900.005693	6.307000e-27	0.005030	0.0470	0.065	1096.0334	0.62
2	6	2	2900.022027	3.048000e-27	0.022620	0.0460	0.060	1593.6378	0.61
3	6	1	2900.027223	1.891000e-25	0.000465	0.0480	0.060	1593.6378	0.61
4	6	2	2900.035027	1.905000e-25	0.000465	0.0480	0.060	1593.6378	0.61
...	...	...	...	...	...	...	...	...	...
41623	6	2	3299.877822	5.946000e-29	0.000004	0.0380	0.061	1416.5543	0.61
41624	6	1	3299.900527	7.204000e-29	0.000221	0.0589	0.077	532.9581	0.75
41625	6	3	3299.901848	2.838000e-25	0.035670	0.0470	0.099	1526.2146	0.75
41626	6	1	3299.984795	5.343000e-29	0.000730	0.0380	0.060	1594.0043	0.61
41627	6	2	3299.989099	5.343000e-29	0.000730	0.0380	0.060	1594.0043	0.61

41628 rows x 25 columns

→ The quanta expressions are now separated into their parts:

```
[6]: update_hitran.hitran_df[['local_upper_quanta', 'J_up', 'N_up', 'sym_up', 'local_lower_quanta',
```

```
[6]:
```

	local_upper_quanta	J_up	N_up	sym_up	local_lower_quanta	J_low	N_low	sym_low
0	13A2	13	2	F1	12A1	1	12	A1
1	14E	14	3	F1	14F2	3	14	F2
2	17F1	17	47	F1	17F2	2	17	F2
3	13F2	21	13	F2	12F1	3	12	F1
4	12E	25	12	E	12E	2	12	E
5	18 10 E	18	10	E	19 7 E	19	7	E
6	14F2	40	14	F2	14F1	1	14	F1
7	12F1	36	12	F1	12F2	3	12	F2
8	13F1	22	13	F1	12F2	2	12	F2
9	13 8 E	13	8	E	14 1 E	14	1	E

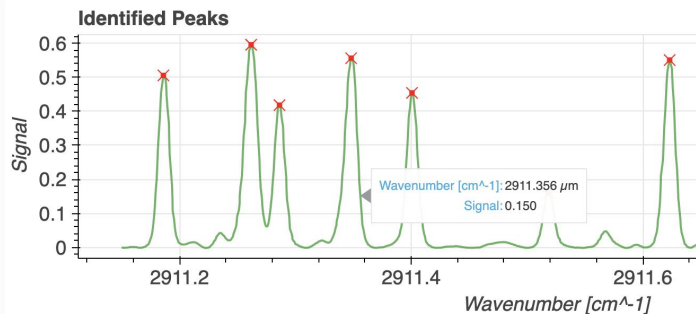
COMPLICATED!

61	2900.158043	2.598E-28	7.865E-03.04300.056	2857.95850.57-.005700	0 2 0 0 1E	0 0 0 0 1A1	12F1 36	12F2 2	364332534338	7 1 7	150.0	150.0
62	2900.160840	4.577E-29	3.967E-02.04400.057	2624.65260.58-.005800	1 0 0 0 1A1	0 0 0 0 1A1	22E 66	23E 1	233332403637	7 1 7	90.0	94.0
61	2900.162815	4.359E-28	7.917E-03.04300.056	2857.95680.57-.005700	0 2 0 0 1E	0 0 0 0 1A1	22F1 96	23F2 2	233332403637	7 1 7	135.0	141.0
61	2900.164396	4.633E-23	8.820E-03.06500.079	62.87570.73-.005800	0 1 0 0 1F1	0 0 0 0 1A1	22F1 59	22F2 3	163332534337	7 1 7	270.0	270.0
							22A1 34	23A2 1	233332403637	7 1 7	225.0	235.0
							4F2 17	3F1 1	444332453638	7 1 7	27.0	21.0

Identifying and Assigning Peaks

→ Automatically find the location of spectral peaks, selecting a minimum height and maximum threshold for the algorithm.

```
[9]: # Find peak centers (wavenumber) and plot over spectrum
peak_centers, peak_heights = assign.line_finder_auto(wavenumber_range = wavenumber_range,
                                                    peak_height_min = 0.05,
                                                    peak_height_max = 0.6,
                                                    __plot__ = True,
                                                    __print__ = True)
```



	peak_centers	peak_heights
0	2911.697485	0.210332
1	2911.676406	0.106295
2	2911.623123	0.571206
3	2911.518359	0.168570
4	2911.400698	0.471979
5	2911.348160	0.572617
6	2911.286321	0.439682
7	2911.261846	0.608428
8	2911.186087	0.524801

→ Find the closest line from HITRAN line list for each peak in the lab spectrum

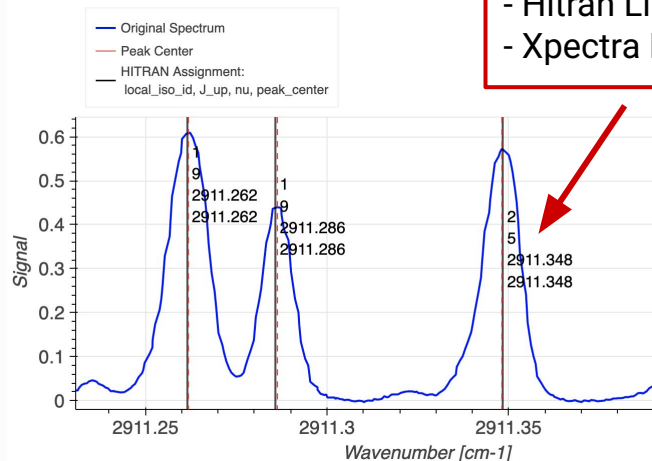
```
[11]: # Filters HITRAN line list
filters = {'local_iso_id' : [1,2]} # Only search for common isotopologue

# Match found lines, plot them over spectrum, and display DataFrame
assign.hitran_line_assigner(threshold = 0.01,
                           filters = filters,
                           columns_to_print = ['local_iso_id', 'J_up', 'nu', 'peak_center'],
                           wavenumber_range = wavenumber_range,
                           __print__ = True, # Display the fitted HITRAN DataFrame
                           __plot_bokeh__ = True, # Plot interactively with Bokeh
                           __plot_seaborn__ = False
                           )
```



BokehJS 3.7.0-dev.1 successfully loaded.

CH4 Spectrum and Found HITRAN Lines



- Isotopologues ID
- J Quantum Number
- Hitran Line Center
- Xpectra Line Center

Spectral Fitting

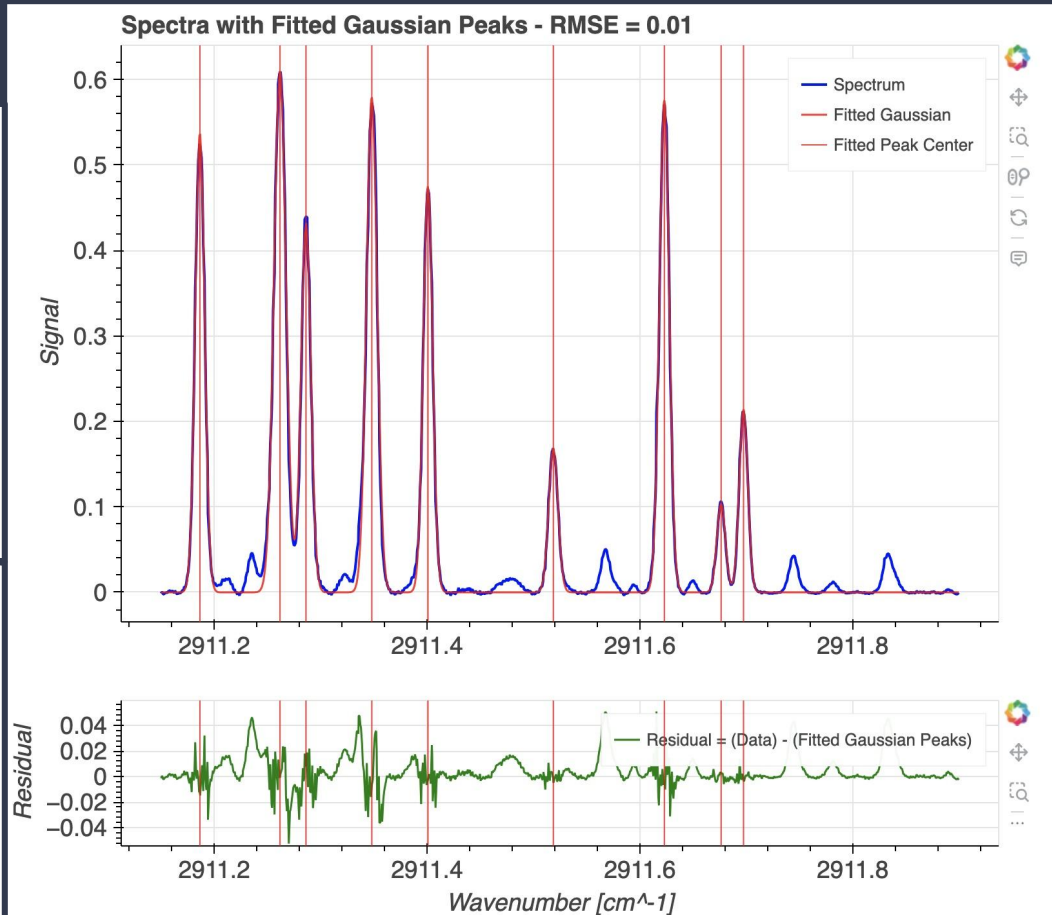
→ Here are the line profile options:

- **Gaussian(x)** = `amplitude * np.exp(-(x - center) ** 2 / (2 * width ** 2))`
- **Lorentzian(x)** = `amplitude / (1 + ((x - center) / width) ** 2)`
- **Voigt(x)** = `amplitude * np.real(wofz(z)).astype(float) / (sigma * np.sqrt(2 * np.pi))`
 - `sigma = wid_g / np.sqrt(2 * np.log(2))`
 - `gamma = wid_l / 2`
 - `z = ((x - center) + 1j * gamma) / (sigma * np.sqrt(2))`

→ Fit spectral lines with Gaussian line profile

```
[10]: specfit.fit_spectrum(initial_guesses,
    line_profile = 'gaussian', # Default
    wavenumber_range = wavenumber_range,
    _plot_bokeh__ = True,
    _print__ = True, # Display fitted params and error bars
)
```

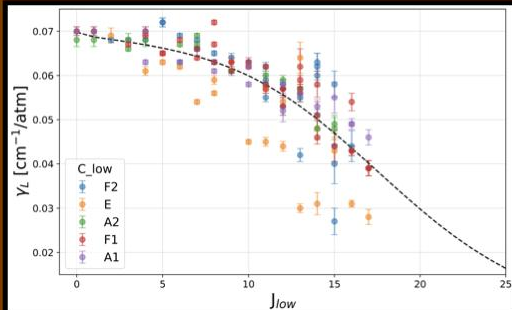
	Peak Number	Center	Center Error	Intensity	Intensity Error	Width	Width Error
0	1	2911.6974	0.0012	0.214	0.0566	0.004	0.0012
1	2	2911.6764	0.0025	0.104	0.0543	0.004	0.0025
2	3	2911.6230	0.0004	0.576	0.0468	0.005	0.0004
3	4	2911.5187	0.0016	0.169	0.0544	0.004	0.0016
4	5	2911.4009	0.0005	0.475	0.0491	0.004	0.0005
5	6	2911.3481	0.0004	0.579	0.0442	0.005	0.0005
6	7	2911.2862	0.0006	0.432	0.0508	0.005	0.0006
7	8	2911.2618	0.0004	0.609	0.0403	0.006	0.0004
8	9	2911.1865	0.0005	0.537	0.0492	0.004	0.0005



Xpectra: Opacity Generator

(4) Pressure Broadening

- Utilize lab coeff.
- Collect/vet literature coeff.



- Fitting all coeff. across:
 - Quantum Numbers
 - Symmetry
 - Band
 - Models (4th Pade Eq.)

(5) Modify HITRAN

- Load/parse HITRAN line list
- Create DataFrame with spectroscopic parameters

molec_id	local_iso_id	nu	sw	gamma_air	gamma_self	J_low	J_up	sym_low
6	2	2900.000621	1.825000e-25	0.0490	0.067	12	13	A1
6	2	2900.005693	6.307000e-27	0.0470	0.065	14	14	F2
6	2	2900.022027	3.048000e-27	0.0460	0.060	17	17	F2
6	1	2900.027223	1.891000e-25	0.0480	0.067	12	13	F1
6	2	2900.035027	1.905000e-25	0.0400	0.067	12	12	E
6	3	2900.036129	1.681000e-28	0.0377	0.056	19	18	E

- Replace HITRAN broadening coefficients with fitted data

(6) Generate Opacity

- Read modified line list
- Implement Hapi library

- Specify:
 - Wavelength range
 - Line profile (e.g., Voigt)
 - Resolution
 - P,T grid (e.g., 1460)
 - Wing cutoff
 - Intensity Cutoff

- Generate cross section

Fit Literature Data

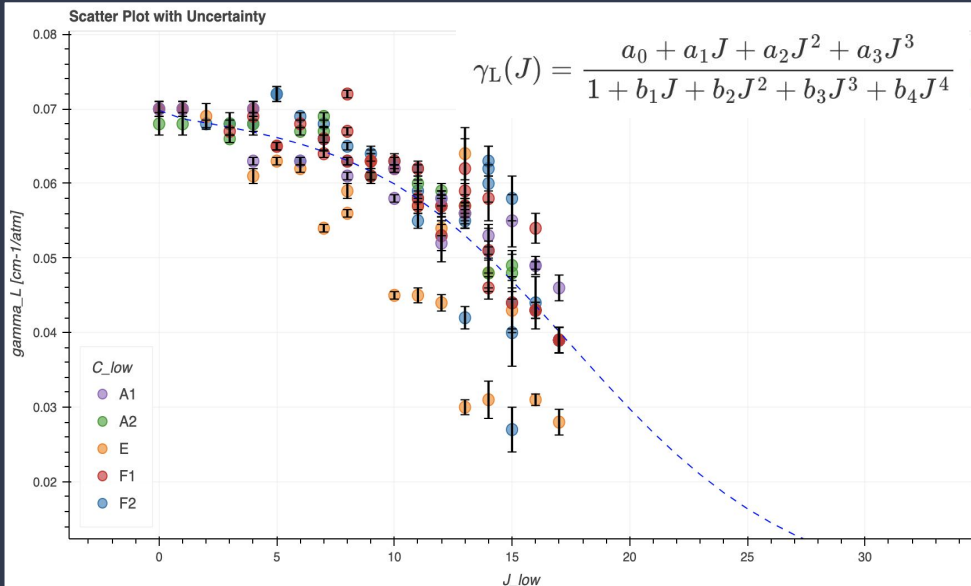
→ Fit a curve to the data using the 4th Pade equation

→ The function plots the fitted curve, prints the fitted parameters, and returns them in an array

```
[13]: # Choose new filters
filters = {
    'gamma_uncertainty': lambda x: x < 0.01, # Maximum uncertainty threshold
    'C_low' : ['A1','A2','E','F1','F2'],
    'author' : ['Gharib-Nezhad','fake'] # Filter to only Gharib-Nezhad and fake data points
}

# Plot J_low vs gamma_L with uncertainty, fitting 4th pade
fit_lit.plot_with_uncertainty(param_to_sort = 'C_low', # Color-code by symmetry
                             filters = filters,
                             fit_4thPade = True,
                             show_plot = True
                             )
```

```
a0: 0.07 +/- 0.003
a1: -713.724 +/- 67744643.277
a2: 35.902 +/- 3407708.755
a3: -0.561 +/- 53214.811
b1: -10310.6 +/- 978654247.181
b2: 437.164 +/- 41494658.868
b3: -3.726 +/- 353718.362
b4: -0.541 +/- 51325.137
```



Update HITRAN Linelist

→ HITRAN line lists can be very large for a given molecule and wavenumber range, often including millions of lines.

→ Because of this, since we want to perform a calculation on each line, we'll use multiprocessing for this task. Parallization decreases the time to process significantly and doesn't overload memory.

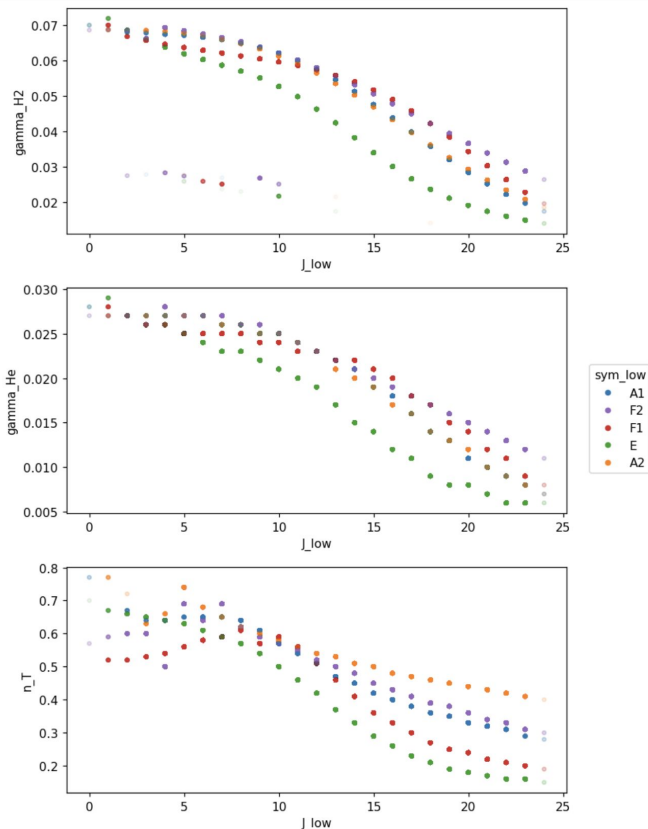
→ We modify each line from the HITRAN file, calculating γ_{H_2} , γ_{He} , and n_T from J_{low} and sym_{low} , and load into a new text file.

```
[12]: # Replace lines in HITRAN file using parallel processing
update_hitran.replace_in_file_parallel(pb_coeffs = pbro,
                                       save_name = 'output_v3.txt',
                                       num_processes=8)
```

```
[15]:
```

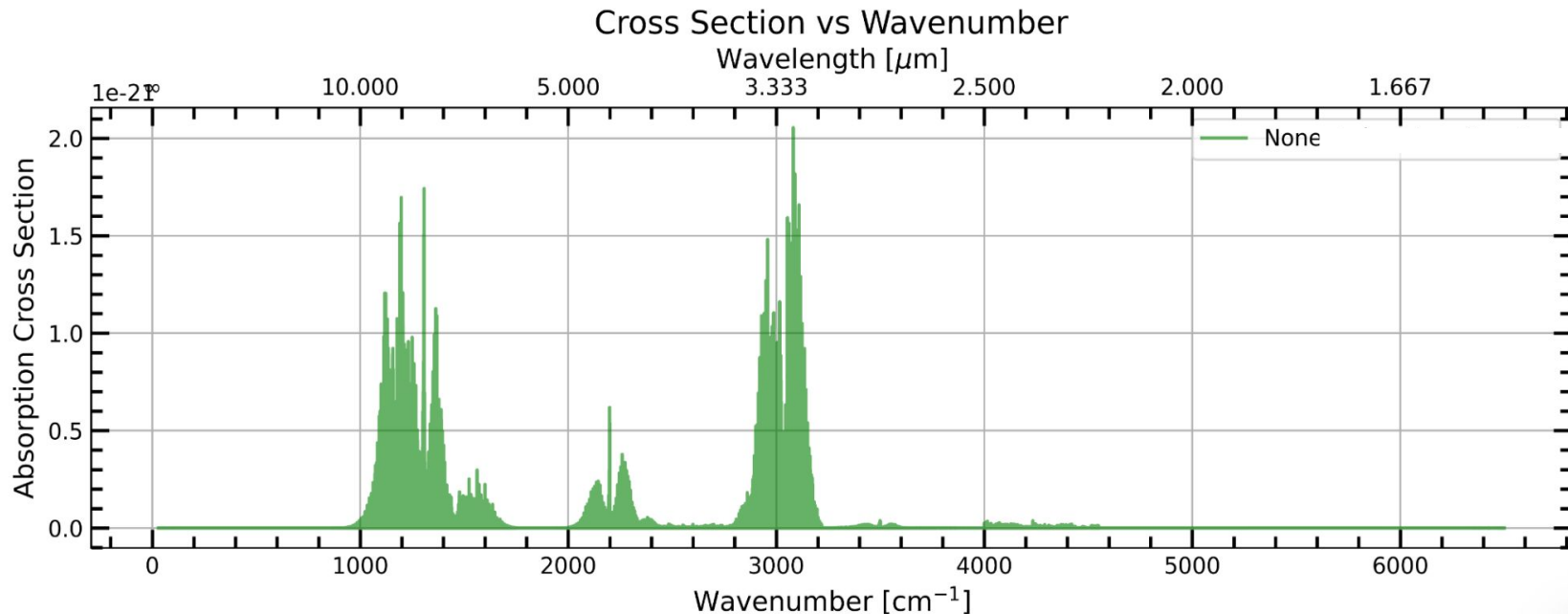
	molec_id	local_iso_id	nu	sw	a	γ_{H_2}	γ_{He}	elower	n_T	deltz
0	6	2	2900.000621	1.825000e-25	0.023890	0.0576	0.023	814.6845	0.51	-0.0
1	6	2	2900.005693	6.307000e-27	0.005030	0.0532	0.021	1096.0334	0.48	-0.0
2	6	2	2900.022027	3.048000e-27	0.022620	0.0450	0.018	1593.6378	0.41	-0.0
3	6	1	2900.027223	1.891000e-25	0.000465	0.0574	0.023	815.1315	0.51	-0.0
4	6	2	2900.035027	1.905000e-25	0.067460	0.0463	0.019	815.0317	0.42	-0.0

5 rows × 25 columns



Opacity Generator (In progress)

```
generate_XS (linelist_name = '12CH3D_modified',  
            res = DeltaWN_1460[i],  
            P_bar = P_1460[i], # in mbar  
            T = T_1460[i], # in Kelvin  
            WN1 = 30.0,  
            WN2 = 6500,  
            Diluent = {'gamma_air':0.85, 'gamma_self':0.15},  
            ind_1460 = i1460[i], # layer  
            Intensity_cutoff = 10e-41,  
            plot_bokeh = True,  
            plot_seaborn = True,  
            save = False  
)
```

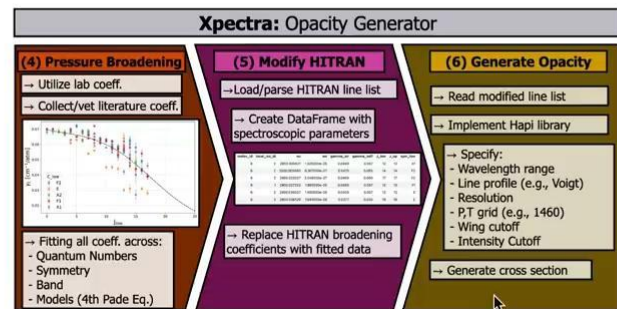
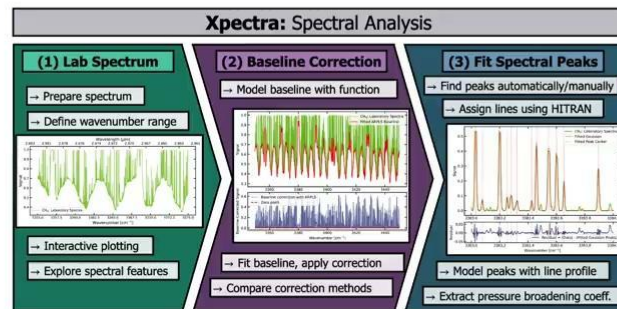


Xpectra Website

- Created using Sphinx
- Installation instructions
- Tutorials
- Documentation
- GitHub

Xpectra

For analyzing exoplanetary spectra in the lab, providing insights into atmospheric composition and habitability. The `Xpectra` workflow is as follows:



Lessons Learned

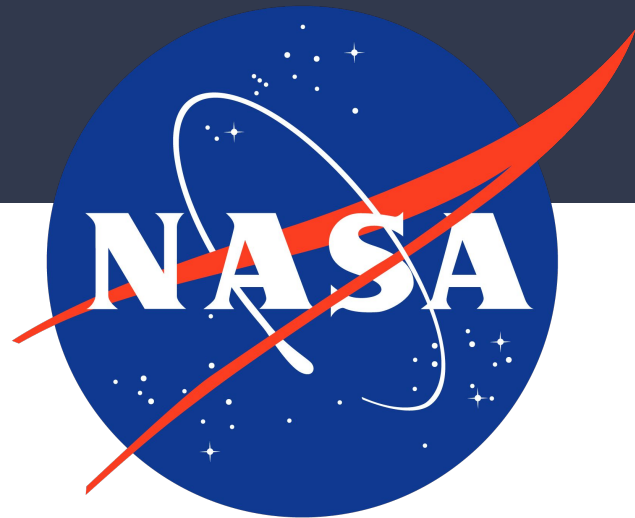
- Contributed to HWO White Paper
- 1st author of the JOSS Paper (in progress)
 - Published my first software package
- My post-internship plans
 - Astrophysics → data science
 - Applying to master's programs
 - Leveraging what I've learned about programming practices, software development, data analysis



NASA Ames Research Center / Astrophysics Laboratory

Conclusion/Outlook

- Continued collaboration with code ST
- In progress
 - ◆ Line profile
 - ◆ Benchmarking
 - ◆ Opacity Generator



Developed as a part of the XRP23 grant: “From Super-Earths to Sub-Neptunes: Laboratory Measurements of High-Temperature Absorption Cross Sections and Pressure-broadening coefficients of key Exoplanetary molecules with High Resolution Mid-IR Spectroscopy” [PI: Bejaoui, Sci-PI: Gharib-Nezhad]

EGN and EM would like to thank OSTEM internships and funding through the NASA, award AL-67060

Baseline fitting:

Asymmetrically reweighted penalized least squares smoothing (ARPLS) Method

Baseline correction methods based on penalized least squares are successfully applied to various spectral analyses. The methods change the weights iteratively by estimating a baseline. If a signal is below a previously fitted baseline, large weight is given. On the other hand, no weight or small weight is given when a signal is above a fitted baseline as it could be assumed to be a part of the peak.

```
# Initialize signal and parameters
y = signal_values # Raw signal data
N = length(y) # Number of data points in the signal
D = difference_matrix(N) # Create difference matrix for smoothing
H = smoothing_penalty * transpose(D) * D # Calculate smoothing term

# Initialize weights
w = array_of_ones(N) # Start with equal weights

# Iterative fitting loop
for i in range(max_iterations):
    W = diagonal_matrix(w) # Create weight matrix
    WH = W + H # Combine weight matrix and smoothing penalty

    # Solve for the estimated baseline using LU decomposition
    z = solve_system(WH, w * y)

    # Calculate difference between the original signal and baseline estimate
    d = y - z

    # Find negative differences (outliers below the baseline)
    dn = find_negatives(d)

    # Update mean and standard deviation of these outliers
    m = mean(dn)
    s = standard_deviation(dn)

    # Update weights based on outlier information
    wt = 1 / (1 + exp(2 * (d - (2 * s - m)) / s))

    # Check if the change in weights is small enough to stop
    if relative_change(w, wt) < threshold:
        break

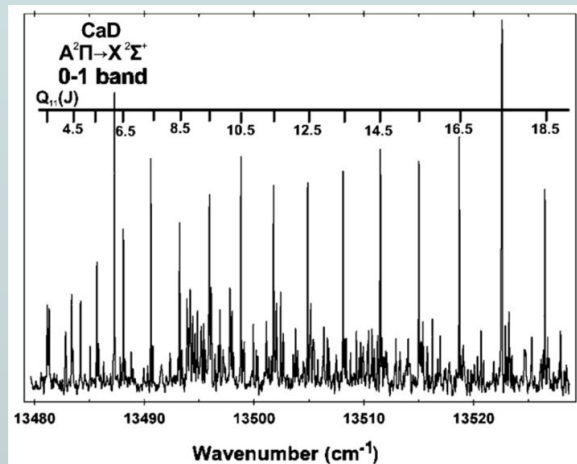
    # Update weights for the next iteration
    w = wt

# Final corrected signal = original signal - estimated baseline
corrected_signal = y - z
```

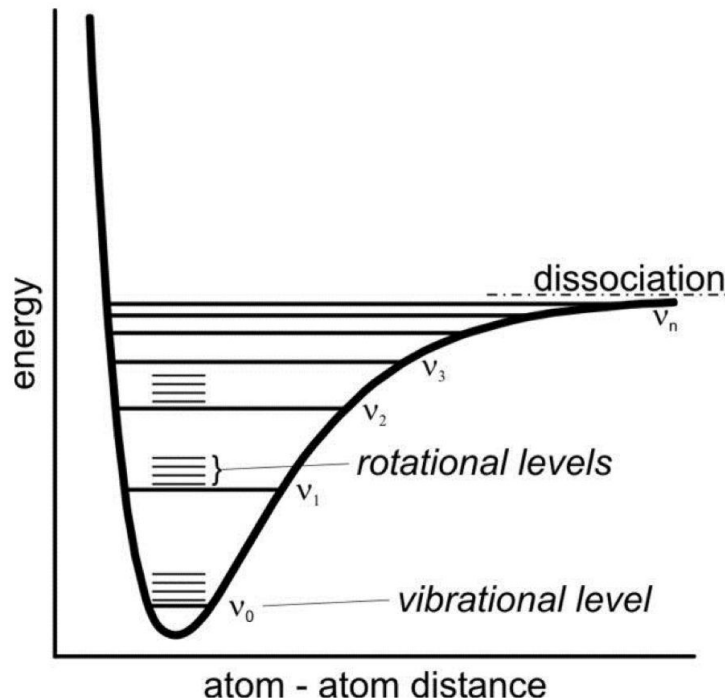
Comments suggested by ChatGPT

From Line List to Opacity: Spectral Line Positions - Energy Levels

X



Gharib-Nezhad + 2012 JMS



Kraft+2006 "Vibrational Spectroscopic Sensors"

Line List to Opacity: Line Profiles

X

Doppler

The thermal translational motion of the spectroscopically active molecule at velocity v leads to frequency shift of $\Delta\nu = \pm (va/c) \nu_0$ known as Doppler effect.

$$f_D(\nu - \nu_0) = \frac{1}{\gamma_G \sqrt{\pi}} \exp\left(-\frac{(\nu - \nu_0)^2}{\gamma_G^2}\right)$$

$$\gamma_G = \sqrt{\frac{2k_B T}{m}} \frac{\nu_0}{c}$$

Lorentz

Individual collisions of molecules lead to energy exchanges between radiators and perturbers. These exchanges shorten the lifetime of the initial and final states of the optical transition and yield what is called pressure or collisional broadening.

$$F_L(\nu - \nu_0) = \frac{1}{\pi} \frac{\gamma_L}{(\nu - \nu_0)^2 + \gamma_L^2}$$

$$\gamma_L = \left(\frac{T_{\text{ref}}}{T}\right)^n P \sum_b \gamma_{L,b} p_b$$

Voigt

Convolution of the Gaussian and Lorentzian profiles, given as

$$f_V(\nu - \nu_0) = \int_{-\infty}^{\infty} f_G(\nu' - \nu_0) f_L(\nu - \nu') d\nu'$$

Acronym	Profile name	Parameters		Mechanism	
		N		SD ^a	VC ^a
DP	Doppler	1	Γ_D	No	No
LP	Lorentz	2	Γ, Δ	No	No
VP	Voigt	3	Γ_D, Γ, Δ	No	No
GP	Galatry	4	$\Gamma_D, \Gamma, \Delta, \nu_{VC}$	No	Soft
RP	Rautian	4	$\Gamma_D, \Gamma, \Delta, \nu_{VC}$	No	Hard
NGP	Nelkin–Ghatak	4	$\Gamma_D, \Gamma, \Delta, \nu_{VC}$	No	Hard
SDVP ^b	Speed-dependent Voigt	5	$\Gamma_D, \Gamma_0, \Delta_0, \Gamma_2, \Delta_2$	Yes	No
SDGP ^b	Speed-dependent Galatry	6	$\Gamma_D, \Gamma_0, \Delta_0, \Gamma_2, \Delta_2, \nu_{VC}$	Yes	Soft
SDNGP ^b	Speed-dependent Nelkin–Ghatak	6	$\Gamma_D, \Gamma_0, \Delta_0, \Gamma_2, \Delta_2, \nu_{VC}$	Yes	Hard
SDRP ^b	Speed-dependent Rautian	6	$\Gamma_D, \Gamma_0, \Delta_0, \Gamma_2, \Delta_2, \nu_{VC}$	Yes	Hard
HTP	Hartmann–Tran	7	$\Gamma_D, \Gamma_0, \Delta_0, \Gamma_2, \Delta_2, \nu_{VC}, \eta$	Yes	Hard
CSDaRSP ^b	Correlated SD asymmetric Rautian–Sobelman	8	$\Gamma_D, \Gamma_0, \Delta_0, \Gamma_2, \Delta_2, \nu_{VC}, \chi, \eta$	Yes	Combination
pCSDKS ^b	Partially correlated SD Keilson-Storer	8	$\Gamma_D, \Gamma_0, \Delta_0, \Gamma_2, \Delta_2, \nu_{VC}, \gamma_{KS}, \eta$	Yes	Combination